LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 10 – PENCARIAN NILAI MAX/MIN

I Kadek Indra Mahottama

109082500039

KELAS S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pencarian nilai ekstrim (maksimum atau minimum) merupakan salah satu algoritma fundamental dalam pemrograman yang memproses sekumpulan data array secara sekuensial. Konsep logika dari algoritma ini sangat sederhana, yaitu dimulai dengan mengasumsikan elemen pertama dari suatu himpunan data sebagai kandidat nilai ekstrim awal. Selanjutnya, program akan melakukan iterasi atau perulangan untuk memvalidasi kandidat tersebut dengan cara membandingkannya terhadap elemen-elemen berikutnya secara berturut-turut hingga mencapai data terakhir. Apabila dalam proses komparasi tersebut ditemukan elemen data yang lebih memenuhi kriteria (misalnya bernilai lebih besar untuk pencarian maksimum), maka status kandidat nilai ekstrim akan diperbarui dengan data yang baru. Pada akhir iterasi, nilai yang berhasil dipertahankan sebagai kandidat akan dipastikan sebagai nilai ekstrim yang valid dari keseluruhan kumpulan data tersebut.

Implementasi pencarian nilai ekstrim ini memiliki sedikit penyesuaian tergantung pada kompleksitas struktur data yang digunakan. Pada array yang berisi tipe data dasar seperti bilangan bulat, algoritma dapat beroperasi dengan langsung menyimpan dan memperbarui nilai aslinya di dalam variabel penampung. Namun, pada kasus komputasi yang lebih kompleks menggunakan tipe data terstruktur atau struct (seperti mencari identitas mahasiswa dengan nilai IPK tertinggi atau pembalap dengan waktu tercepat), pendekatan yang paling tepat adalah dengan memproses dan menyimpan nomor indeks dari data tersebut, bukan sekadar nilai propertinya. Dengan mempertahankan indeks array dari data yang memiliki nilai ekstrim, sistem tidak hanya mendapatkan besaran nilainya, tetapi juga mampu memanggil dan menampilkan seluruh atribut informasi lain yang terikat pada identitas pemegang nilai ekstrim tersebut secara utuh.

B. Guided

1. [Mencari nilai terkecil]

```
package main

import "fmt"

type arrInt [2023]int

func terkecil_1(tabInt arrInt, n int) int {
    var min int = tabInt[0]
    var j int = 1

    for j < n {
```

```
        if min > tabInt[j] {
            min = tabInt[j]
        }
        j = j + 1
    }
    return min
}

func terkecil_2(tabInt arrInt, n int) int {
    var idx int = 0
    var j int = 1

    for j < n {
        if tabInt[idx] > tabInt[j] { // [9, 8, 5, 6]
            idx = j
        }
        j = j + 1
    }
    return idx
}

func main() {
    var n int
    var arr arrInt

    fmt.Print("Masukkan jumlah data: ")
    fmt.Scan(&n)

    fmt.Println("Masukkan elemen array :")

    for i := 0; i < n; i++ {
        fmt.Scan(&arr[i])
    }

    min := terkecil_1(arr, n)
    idx := terkecil_2(arr, n)

    fmt.Println("Nilai terkecil: ", min)
    fmt.Println("Indeks nilai terkecil: ", idx)
}
```

Output :

```
PS D:\Coding\Project latihan\GO\Semester 2\Praktikum\109082500039_I-Kadek-indra-Mahottama> go run "d:\C
oding\Project latihan\GO\Semester 2\Praktikum\109082500039_I-Kadek-indra-Mahottama\Week 9 - Modul 10\Gu
ided\1\1.go"
Masukkan jumlah data: 5
Masukkan elemen array :
40
20
60
10
36
Nilai terkecil:  10
Indeks nilai terkecil:  3
```

2. [Mencari data IPK Terkecil dan Terbesar]

```go
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim  string
    IPK  float64
}

type arrMahasiswa [3]mahasiswa

//data index 0 = nama, nim, IPK
//data index 1 = nama, nim, ipk
//data index 2 = nama, nim, ipk

func IPKTerbesar(array []mahasiswa) (mahasiswa, int) {
    var mhsTerbesar mahasiswa = array[0] //nilai awal
    var idxDitemukan int = 0
    var i int

    for i = 1; i < len(array); i++ {
        if array[i].IPK > mhsTerbesar.IPK {
            mhsTerbesar = array[i]
            idxDitemukan = i
        }
    }

    return mhsTerbesar, idxDitemukan
}

func IPKTerkecil(array []mahasiswa) (mahasiswa, int) {
    var mhsTerkecil mahasiswa = array[0] //nilai awal
    var idxDitemukan int = 0
    var i int

    for i = 1; i < len(array); i++ {
```

```
        if array[i].IPK < mhsTerkecil.IPK {
            mhsTerkecil = array[i]
            idxDitemukan = i
        }
    }

    return mhsTerkecil, idxDitemukan
}

func main() {
    var arrMhs arrMahasiswa
    var i int

    for i = 0; i < len(arrMhs); i++ {
        fmt.Println("INPUT DATA MAHASISWA INDEX KE-", i)
        fmt.Print("Masukkan nama : ")
        fmt.Scan(&arrMhs[i].nama)
        fmt.Print("Masukkkan nim : ")
        fmt.Scan(&arrMhs[i].nim)
        fmt.Print("Masukkan IPK : ")
        fmt.Scan(&arrMhs[i].IPK)
        fmt.Println("==================================")
    }
    fmt.Println()

    var mhsMaks, mhsMin mahasiswa
    var idxMaks, idxMin int

    mhsMaks, idxMaks = IPKTerbesar(arrMhs[:])

    mhsMin, idxMin = IPKTerkecil(arrMhs[:])

    fmt.Println("DATA IPK TERKECIL DAN TERBESAR")
    fmt.Println("=== IPK TERBESAR ===")
    fmt.Println("IPK Terbesar = ", mhsMaks.IPK)
    fmt.Println("Atas nama = ", mhsMaks.nama)
    fmt.Println("nim = ", mhsMaks.nim)
    fmt.Println("ditemukan pada indeks ke-", idxMaks)
    fmt.Println()
    fmt.Println("=== IPK TERKECIL ===")
    fmt.Println("IPK Terkecil = ", mhsMin.IPK)
    fmt.Println("Atas nama = ", mhsMin.nama)
    fmt.Println("nim = ", mhsMin.nim)
    fmt.Println("ditemukan pada indeks ke-", idxMin)
}
```

Output :

```
PS D:\Coding\Project latihan\GO\Semester 2\Praktikum\109082500039_I-Kadek-indra-Mahottama> go run "d:\Coding\Project latihan\GO\Semester 2\Praktikum\109082500039_I-Kadek-indra-Mahottama\Week 9 - Modul 10\Guided\2\2.go"
INPUT DATA MAHASISWA INDEX KE- 0
Masukkan nama : Indra
Masukkkan nim : 121
Masukkan IPK : 3.88
================================
INPUT DATA MAHASISWA INDEX KE- 1
Masukkan nama : Kadek
Masukkkan nim : 122
Masukkan IPK : 3.94
================================
INPUT DATA MAHASISWA INDEX KE- 2
Masukkan nama : Mahottama
Masukkkan nim : 123
Masukkan IPK : 3.98
================================

DATA IPK TERKECIL DAN TERBESAR
=== IPK TERBESAR ===
IPK Terbesar =  3.98
Atas nama =  Mahottama
nim =  123
ditemukan pada indeks ke- 2

=== IPK TERKECIL ===
IPK Terkecil =  3.88
Atas nama =  Indra
nim =  121
ditemukan pada indeks ke- 0
```

C. Unguided

1. Soal Unguided 1

   Sebuah program digunakan untuk mendata berat anak kelinci yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat anak kelinci yang akan dijual.

   Masukan terdiri dari sekumpulan bilangan, yang mana bilangan pertama adalah bilangan bulat N yang menyatakan banyaknya anak kelinci yang akan ditimbang beratnya. Selanjutnya N bilangan riil berikutnya adalah berat dari anak kelinci yang akan dijual. Keluaran terdiri dari dua buah bilangan riil yang menyatakan berat kelinci terkecil dan terbesar.

   Jawaban :

```
package main

import "fmt"

type arrKelinci [1000]float64

func main() {
```

```
        var n int
        var berat arrKelinci
        var min, max float64

        fmt.Print("Jumlah kelinci: ")
        fmt.Scan(&n)

        for i := 0; i < n; i++ {
            fmt.Print("berat kelinci ke-",i+1," :")
            fmt.Scan(&berat[i])
        }

        if n > 0 {
            min = berat[0]
            max = berat[0]

            for i := 1; i < n; i++ {
                if berat[i] < min {
                    min = berat[i]
                }
                if berat[i] > max {
                    max = berat[i]
                }
            }

            fmt.Printf("berat terkecil: %v, berat terbesar:
    %v", min, max)
        }
    }
```

Output :

```
PS D:\Coding\Project latihan\GO\Semester 2\Praktikum\109082500039_I-Kadek-indra-Mahottama> go run "d:\C
oding\Project latihan\GO\Semester 2\Praktikum\109082500039_I-Kadek-indra-Mahottama\Week 9 - Modul 10\Un
guided\1\1.go"
Jumlah kelinci: 5
berat kelinci ke-1 :1.2
berat kelinci ke-2 :2.5
berat kelinci ke-3 :0.8
berat kelinci ke-4 :1.5
berat kelinci ke-5 :3.0
berat terkecil: 0.8, berat terbesar: 3
```

Penjelasan :

Pada program ini saya membuat sebuah type array bernama arrKelinci dengan kapasitas seribu elemen bertipe float64 untuk menampung data berat. Di awal jalannya program saya menginstruksikan agar sistem meminta user memasukkan total jumlah kelinci lalu menyimpannya ke dalam variabel n.

Setelah itu saya menggunakan perulangan for agar user bisa menginputkan berat setiap kelinci satu per satu langsung ke dalam indeks array tersebut. Untuk mencari nilai ekstrimnya saya memberikan validasi awal untuk memastikan n lebih besar dari nol lalu saya melakukan inisialisasi dengan menetapkan elemen pertama di indeks ke nol sebagai nilai kandidat untuk variabel min dan max. Langkah selanjutnya adalah melakukan iterasi array mulai dari indeks ke satu sampai data terakhir untuk melakukan perbandingan kondisional. Jika saat iterasi saya menemukan nilai elemen yang lebih kecil dari min maka saya menimpa nilai min tersebut dengan data yang baru dan saya juga menerapkan logika serupa untuk memperbarui nilai max apabila menemukan elemen yang lebih besar. Sebagai eksekusi akhir saya menyuruh program untuk menampilkan hasil perhitungan variabel min dan max yang sudah tervalidasi tersebut ke layar terminal.

2. Soal Unguided 2

   Sebuah program digunakan untuk menentukan tarif ikan yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat ikan yang akan dijual.
   Masukan terdiri dari dua baris, yang mana baris pertama terdiri dari dua bilangan bulat x dan y. Bilangan x menyatakan banyaknya ikan yang akan dijual, sedangkan y adalah banyaknya ikan yang akan dimasukan ke dalam wadah.
   Baris kedua terdiri dari sejumlah x bilangan riil yang menyatakan banyaknya ikan yang akan dijual.
   Keluaran terdiri dari dua baris. Baris pertama adalah kumpulan bilangan riil yang menyatakan total berat ikan disetiap wadah (jumlah wadah tergantung pada nilai x dan y, urutan ikan yang dimasukan ke dalam wadah sesuai urutan pada masukan baris ke-2). Baris kedua adalah sebuah bilangan riil yang menyatakan berat rata-rata ikan di setiap wadah.
   Jawaban :

```
package main

import "fmt"

type arrIkan [1000]float64

func main() {
    var x, y int
    var ikan arrIkan
```

```
    fmt.Print("Masukan jumlah ikan dijual dan banyak
ikan didalam wadah: ")
    fmt.Scan(&x, &y)

    fmt.Print("berat ikan yang akan dijual: ")
    for i := 0; i < x; i++ {
        fmt.Scan(&ikan[i])
    }

    if x > 0 && y > 0 {
        var totalSemua float64
        var jumlahWadah int

        for i := 0; i < x; i += y {
            var beratWadah float64

            for j := 0; j < y && i+j < x; j++ {
                beratWadah += ikan[i+j]
            }

            fmt.Printf("%.2f  ", beratWadah)

            totalSemua += beratWadah
            jumlahWadah++
        }
        fmt.Println()

        rataRata := totalSemua / float64(jumlahWadah)
        fmt.Printf("berat rata rata : %.2f \n",
rataRata)
    }
}
```

Output :

```
PS D:\Coding\Project latihan\GO\Semester 2\Praktikum\109082500039_I-Kadek-indra-Mahottama> go run "d:\C
oding\Project latihan\GO\Semester 2\Praktikum\109082500039_I-Kadek-indra-Mahottama\Week 9 - Modul 10\Un
guided\2\2.go"
Masukan jumlah ikan dijual dan banyak ikan didalam wadah: 5 2
berat ikan yang akan dijual: 1.0 1.5 2.0 1.0 3.5
2.50  3.00  3.50
berat rata rata : 3.00
```

Penjelasan :

Pada Program ini saya mendefinisikan sebuah type array bernama arrIkan yang berkapasitas seribu elemen float64 untuk menyimpan data berat ikan. Langkah pertama yang saya lakukan adalah meminta user menginputkan variabel x sebagai total ikan yang dijual beserta variabel y sebagai batas kapasitas

maksimal per wadahnya. Setelah itu saya menggunakan perulangan for biasa agar user bisa memasukkan angka berat masing masing ikan secara berurutan ke dalam indeks array. Inti dari pemrosesan datanya berada pada teknik nested loop yang saya terapkan dimana perulangan utamanya saya atur agar indeksnya melompat sebanyak nilai y untuk menandakan perpindahan wadah. Di dalam blok perulangan tersebut terdapat perulangan kedua yang bertugas mengakumulasi berat elemen elemen ikan secara spesifik untuk wadah yang sedang dihitung agar nilainya tidak saling tertukar. Begitu hasil penjumlahan berat satu wadah didapatkan dan dicetak ke layar saya langsung menyimpannya ke dalam variabel penampung totalSemua sekaligus menaikkan nilai konter pada variabel jumlahWadah. Pada tahap akhir komputasi saya tinggal menghitung nilai rata rata keseluruhan dengan membagi totalSemua menggunakan jumlahWadah yang sudah dikonversi sementara ke tipe data float64 lalu menampilkan hasil akhir dari kalkulasi tersebut ke layar terminal user.

3. Soal Unguided 3

   Pos Pelayanan Terpadu (posyandu) sebagai tempat pelayanan kesehatan perlu mencatat data berat balita (dalam kg). Petugas akan memasukkan data tersebut ke dalam array. Dari data yang diperoleh akan dicari berat balita terkecil, terbesar, dan reratanya. Buatlah program dengan spesifikasisubprogram sebagai berikut:

```
type arrBalita [100]float64

func hitungMinMax(arrBerat arrBalita; bMin, bMax *float64) {
/* I.S. Terdefinisi array dinamis arrBerat
   Proses: Menghitung berat minimum dan maksimum dalam array
   F.S. Menampilkan berat minimum dan maksimum balita */
     ...
}

function rerata (arrBerat arrBalita) real {
/* menghitung dan mengembalikan rerata berat balita dalam array */
     ...
}
```

   Jawaban :

```
package main

import "fmt"

type arrBalita [100]float64
```

```
func hitungMinMax(arrBerat arrBalita, n int, bMin, bMax
*float64) {
    *bMin = arrBerat[0]
    *bMax = arrBerat[0]

    for i := 1; i < n; i++ {
        if arrBerat[i] < *bMin {
            *bMin = arrBerat[i]
        }
        if arrBerat[i] > *bMax {
            *bMax = arrBerat[i]
        }
    }
}

func rerata(arrBerat arrBalita, n int) float64 {
    var total float64
    for i := 0; i < n; i++ {
        total += arrBerat[i]
    }
    return total / float64(n)
}

func main() {
    var n int
    var berat arrBalita
    var min, max float64

    fmt.Print("Masukan banyak data berat balita : ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Printf("Masukan berat balita ke-%d: ", i+1)
        fmt.Scan(&berat[i])
    }

    if n > 0 {
        hitungMinMax(berat, n, &min, &max)
        rataRata := rerata(berat, n)

        fmt.Printf("Berat balita minimum: %.2f kg\n",
min)
        fmt.Printf("Berat balita maksimum: %.2f kg\n",
max)
        fmt.Printf("Rerata berat balita: %.2f kg\n",
rataRata)
    }
}
```

Output :

```
PS D:\Coding\Project latihan\GO\Semester 2\Praktikum\109082500039_I-Kadek-indra-Mahottama> go run "d:\C
oding\Project latihan\GO\Semester 2\Praktikum\109082500039_I-Kadek-indra-Mahottama\Week 9 - Modul 10\Un
guided\3\3.go"
Masukan banyak data berat balita : 4
Masukan berat balita ke-1: 5.3
Masukan berat balita ke-2: 6.2
Masukan berat balita ke-3: 4.1
Masukan berat balita ke-4: 9.9
Berat balita minimum: 4.10 kg
Berat balita maksimum: 9.90 kg
Rerata berat balita: 6.38 kg
```

Penjelasan :

Pada kode ini saya mendefinisikan sebuah type array bernama arrBalita dengan kapasitas seratus data bertipe float64. Di dalam fungsi utama saya pertama tama meminta user untuk memasukkan jumlah data balita yang kemudian saya simpan ke variabel n. Setelah itu saya menyusun perulangan for agar user dapat menginputkan angka berat masing masing balita secara berurutan langsung ke dalam indeks array. Untuk proses kalkulasinya saya memecah program menjadi dua fungsi terpisah yaitu fungsi hitungMinMax dan fungsi rerata. Bagian yang cukup teknis ada pada fungsi hitungMinMax dimana saya secara sengaja memanfaatkan mekanisme pointer untuk parameter bMin dan bMax sehingga setiap proses pembaruan nilai kandidat minimum dan maksimum di dalam iterasi akan langsung merubah alamat memori variabel aslinya di fungsi utama tanpa perlu melakukan return. Sementara itu untuk fungsi rerata saya hanya melakukan perulangan biasa untuk mengakumulasi total berat yang kemudian langsung saya bagi dengan jumlah data hasil konversi float64. Sebagai penutup eksekusi saya memanggil kedua fungsi tersebut secara berurutan lalu mencetak variabel penampung hasilnya ke layar terminal dengan format presisi dua angka desimal.

D. Kesimpulan

Praktikum minggu ini memberikan pemahaman komprehensif mengenai implementasi algoritma pencarian nilai ekstrim (maksimum dan minimum) pada struktur data array secara sekuensial. Melalui tiga latihan koding yang telah diselesaikan, kita tidak hanya mempraktikkan konsep dasar penetapan kandidat awal dan komparasi iteratif untuk mencari batasan nilai suatu himpunan data, tetapi juga berhasil mengembangkan kemampuan manipulasi perulangan tingkat lanjut, seperti penggunaan nested loop dengan lompatan indeks untuk memproses dan mengelompokkan data ke dalam beberapa wadah. Lebih dari itu, praktikum ini berhasil mengintegrasikan materi array dengan penggunaan

fungsi dan pointer, yang membuktikan bahwa pemrosesan algoritma yang kompleks dapat dipecah menjadi subprogram yang efisien dan mampu memodifikasi beberapa nilai sekaligus secara langsung di dalam memori. Penguasaan kombinasi teknik-teknik fundamental ini sangat krusial untuk membangun logika pengolahan data angka yang terstruktur, dinamis, dan optimal di dalam bahasa pemrograman Go.